

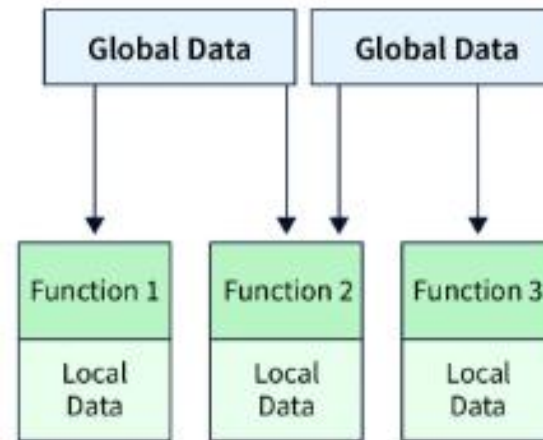
Einführung Projekt OOP

Workshop OO Modellierung

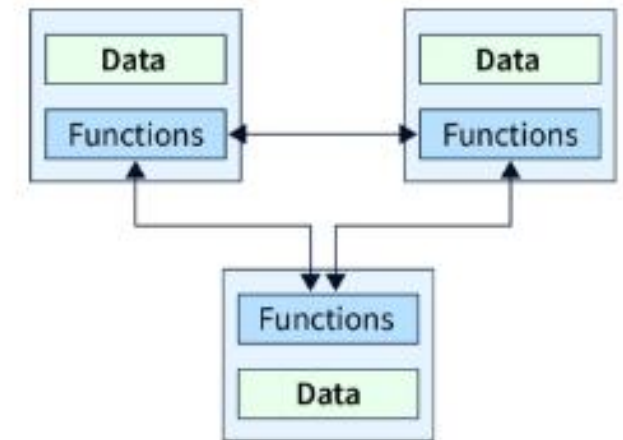
Dr. Philipp Hurni, Kantonsschule Sursee

Ziel des Projekts

Procedural Oriented Programming

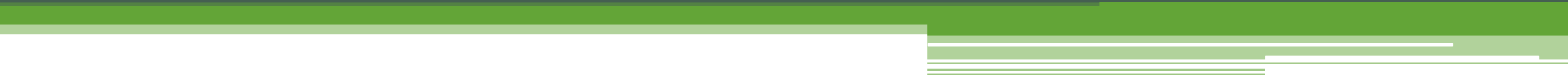


Object Oriented Programming

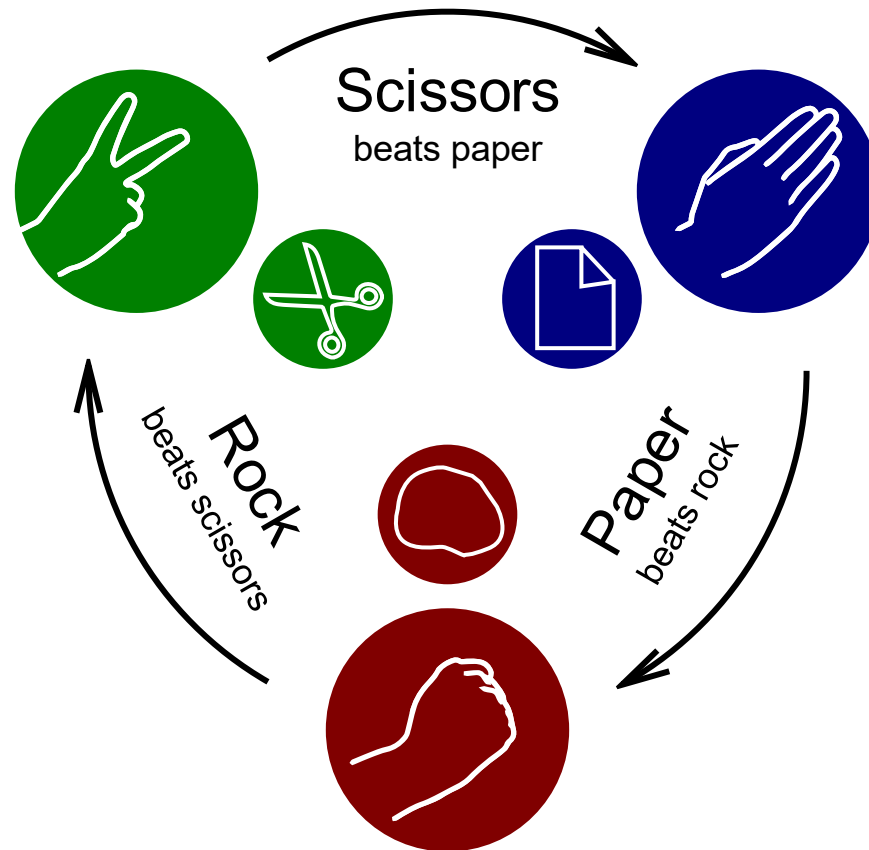


- Objektorientiert Denken
 - Eine eigene Problemstellung definieren
 - Ein eigenes Klassen- und Objektmodell daraus ableiten
 - Ein eigenes Programm daraus ableiten

Beispiel - Problemstellung



DSpiel «Schere Stein Papier» programmieren



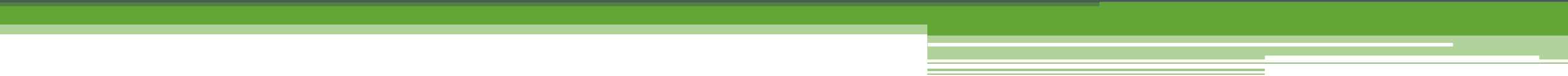
Schere Stein Papier

Schere, Stein, Papier ist ein einfaches Handspiel für zwei Spieler, das oft als Entscheidungsfindung genutzt wird.

Beide Personen zählen gemeinsam bis Drei und formen gleichzeitig mit einer Hand ein Symbol für Schere (Zeige- und Mittelfinger), Stein (Faust) oder Papier (flache Hand). Es gilt das Prinzip, dass immer eines gegen ein anderes gewinnt, gegen das andere Symbol aber verliert. Schere gewinnt gegen Papier (schneiden), Papier gewinnt gegen Stein (einwickeln), und Stein gewinnt gegen Schere (abstumpfen).

Identische Symbole führen zu einem Unentschieden und das Spiel wird wiederholt. Meistens werden Punkte gezählt, d.h. jeder Durchgang gibt einen Punkt, und wer zuerst bei 10 ist hat gewonnen. Viel Spass!

Klassen identifizieren



Welche Klassen soll ich nun definieren?

- Objektorientierte Modellierung ist ein Abbild ihres eigenen Denkens
- Es gibt nicht die eine perfekte Modellierung. Es gibt immer mehrere Lösungen.
- Eine Klasse sollte möglichst eine einzelne Verantwortung haben (Single Responsibility Principle)
- Zusammengehörige Dinge sollten in einer Klasse definiert sein
- Schlechter Ansatz: eine einzige Klasse die alles macht, andere Klassen sind nur Datenspeicher

Klassen identifizieren

- a) Suchen Sie nach Substantiven im Text
- b) Ist das Substantiv nur ein Wert, der zu einem anderen Objekt gehört?
Dann sollte es keine Klasse werden
- c) Aktion/Verb als Substantiv (Nominalisierung): Beschreibt das Wort eigentlich eine Methode/Handlung? Dann auch nicht
- d) Wertebereich/Aufzählung: Handelt es sich um feste Begriffe ohne eigene Logik?
- e) Ist der Begriff für die Software-Logik irrelevant?

Klassen identifizieren

a) Suchen Sie nach Substantiven im Text

Substantive im Text

Schere	Mittelfinger
Stein	Personen
Papier	Prinzip
Symbol	Punkt
Drei	Spiel
Durchgang	Spass
Entscheidungsfindung	Unentschieden
Faust	Zeigefinger
Hand	Handspiel

Klassen identifizieren

b) Ist das Substantiv nur ein Wert, der zu einem anderen Objekt gehört? Dann sollte es keine Klasse werden

	Substantive im Text		
Sind Ausprägungen des Symbols	Schere	Mittelfinger	Gehört zum Symbol Schere
	Stein	Spieler	
	Papier	Prinzip	
Ist ein Wert bei Spiel	Symbol	Punkt	Ist ein Wert bei Spiel
	Drei	Spiel	
	Durchgang	Spass	
Gehören zum Symbol	Entscheidungsfindung	Unentschieden	Ist ein Wert bei Spiel
	Faust	Zeigefinger	
	Hand	Handspiel	

Klassen identifizieren

c) Aktion / Verb als Substantiv (Nominalisierung): Beschreibt das Wort eigentlich eine Methode/Handlung? Dann auch nicht

Substantive im Text

Spieler	
Prinzip	
Symbol	
Spiel	
Durchgang	Spass
Entscheidungsfindung	
Handspiel	

Klassen identifizieren

- d) Wertebereich / Aufzählung: Handelt es sich um feste Begriffe ohne eigene Logik?
- e) Ist der Begriff für die Software-Logik irrelevant?

Substantive im Text	
	Spieler
	Prinzip
Symbol	
	Spiel
Durchgang	Spass

Verbleibende Klassen

Es bleiben übrig

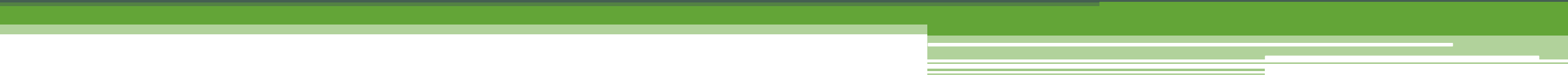
Spieler

Spiel

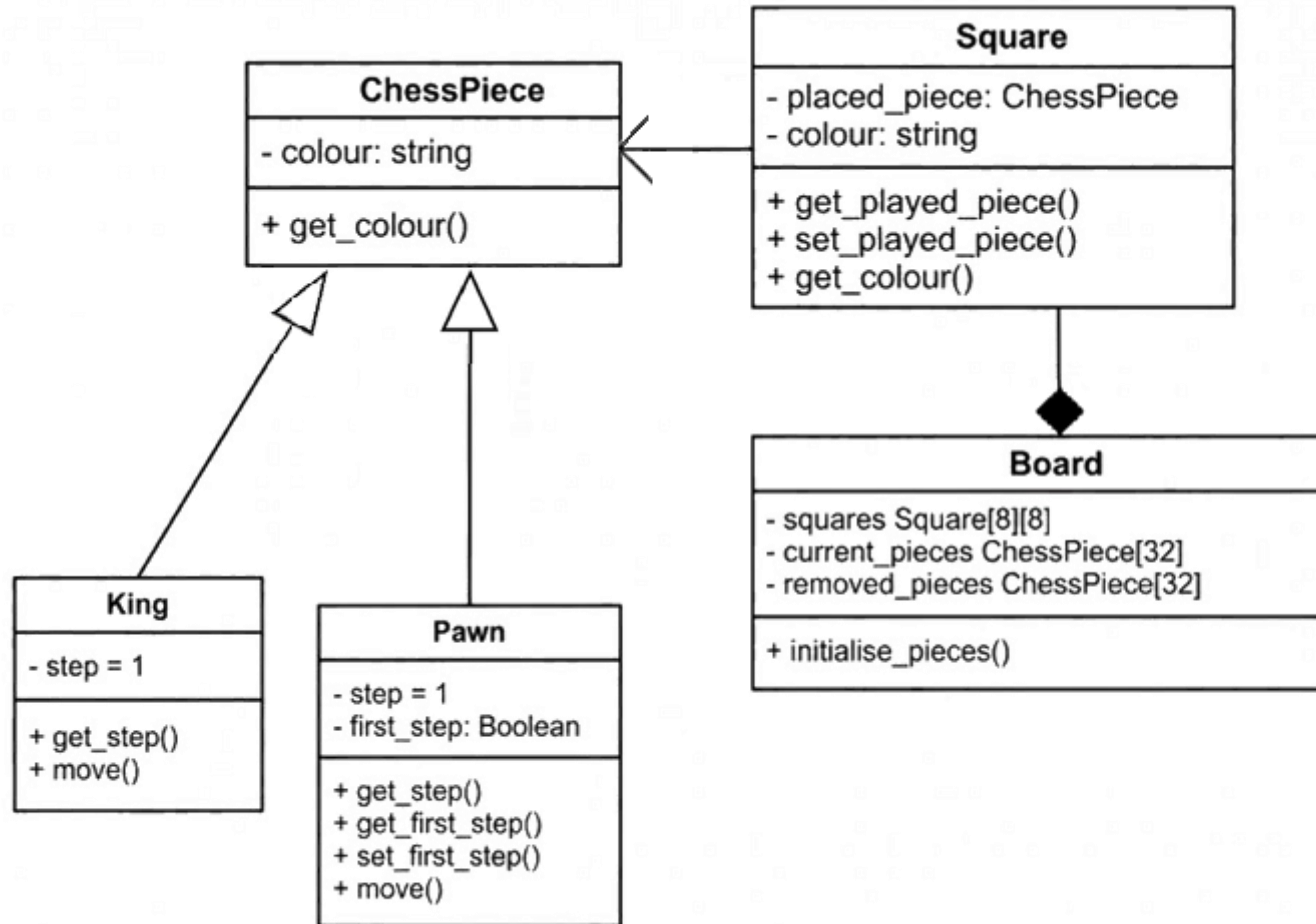
Durchgang

Symbol

Klassendiagramm entwerfen



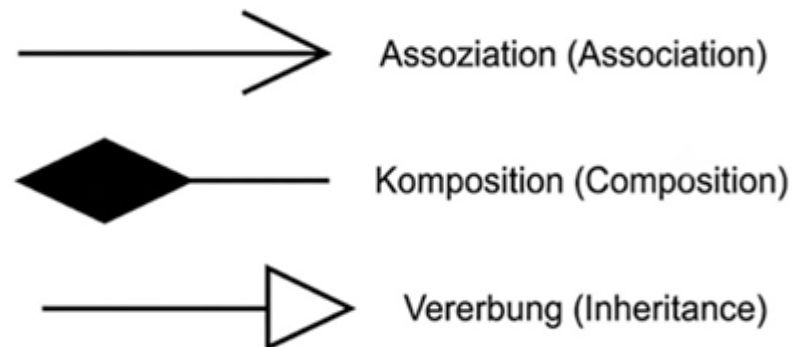
Klassendiagramm

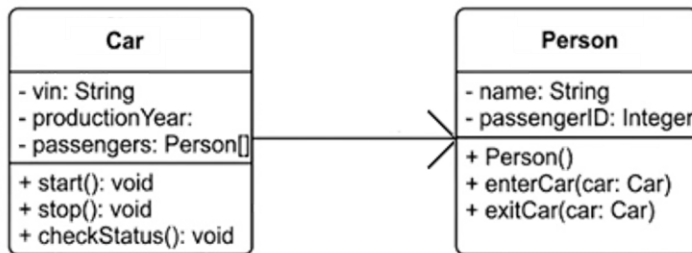


UML – Wichtigste grafische Elemente

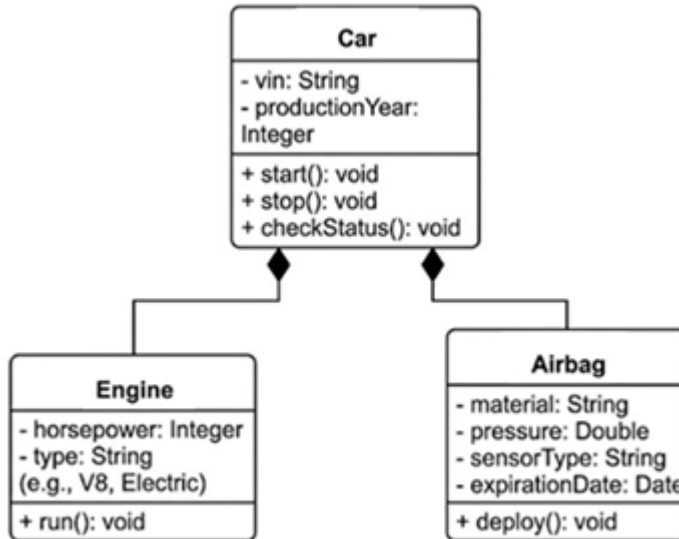
In einem Klassendiagramm werden die Verbindungen zwischen Klassen als Pfeile dargestellt. Variablen, die auf andere Objekte verweisen, werden nicht mit den übrigen Klassenattributen geschrieben, sondern als Verbindungen zwischen den Klassen dargestellt.

Es gibt 6-7 verschiedene Verbindungstypen, wir wollen hier die 3 geläufigsten kurz kennenlernen.

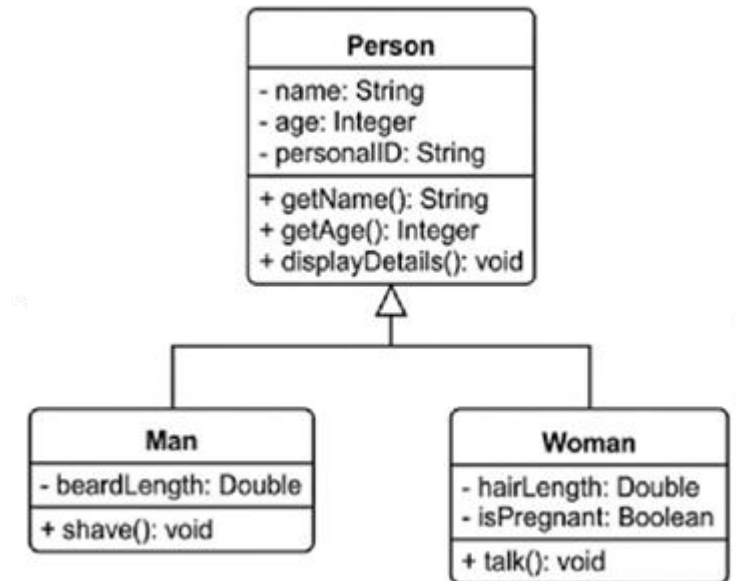




Ein Auto hat 0 oder mehr Passagiere. Diese sind Objekte vom Typ Person. Objekte vom Typ Person sind aber nicht immer in einem Auto und können auch selbständig existieren (machen auch ohne Auto Sinn).

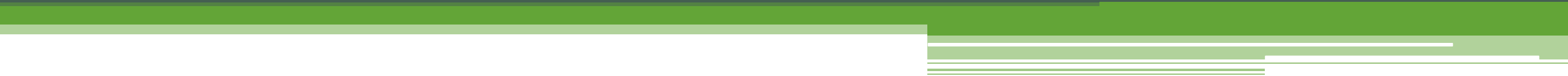


Ein Auto hat einen Motor und hat ein oder mehrere Airbags. Diese Klassen gehören inhärent zum Auto, sind aber austauschbar und haben auch eigene Methoden. Hier nutzt man die Komposition



Ein Mann (Man) ist vom Typ Person, ist aber eine "Spezialisierung" vom Typ Person. Er hat alle Attribute, welche Person hat. Dasselbe gilt für die Klasse Frau (Woman). Sie hat aber andere Attribute, welche Mann nicht hat.

Klassen implementieren



Skelett – Main Klasse kreiert Game

```
public class Main {  
    public static void main(String[] args) {  
  
        System.out.println("starting up Game Skeleton (2-Player)");  
        Game game = new Game();  
        Player firstPlayer = new Player();  
        Player secondPlayer = new Player();  
  
        game.addPlayer(firstPlayer);  
        game.addPlayer(secondPlayer);  
        game.runGame();  
  
        System.out.println("Game ended.");  
    }  
}
```

Ich brauche nun mindestens die folgenden Klassen

```
public class Game {  
    public Game() {  
        System.out.println("Game started...");  
    }  
    public void addPlayer(Player player) {  
    }  
    public void runGame() {  
    }  
}
```

```
import java.util.Scanner;  
  
public class Player {  
    private String name;  
    private Scanner scanner;  
  
    public Player() {  
        this.scanner = new Scanner(System.in);  
    }  
}
```

Aufgabe I: Klassen Kreieren

- a) Kreiere die Klassen
- b) Die Person Klasse soll auf der Konsole nach dem Namen fragen und dann in der Objektvariable speichern

Möglicher Ansatz

```
import java.util.ArrayList;

public class Game {
    private ArrayList<Player> players;

    public Game() {
        System.out.println("Game started...");
        this.players = new ArrayList<Player>();
    }

    public void addPlayer(Player player) {
        this.players.add(player);
    }

    public void runGame() {
        System.out.println("Starting Game");
        for (int i = 0; i < this.players.size(); i++) {
            Player p = this.players.get(i);
            System.out.println("Player " + i + " enter your name:");
            p.askForName();
        }
        for(int round=0; round<10; round++) {
            System.out.println("We are entering round: " + round);
        }
        System.out.println("Game is over");
    }
}
```

```
import java.util.Scanner;

public class Player {
    private String name;
    private Scanner scanner;

    public Player() {
        this.scanner = new Scanner(System.in);
    }

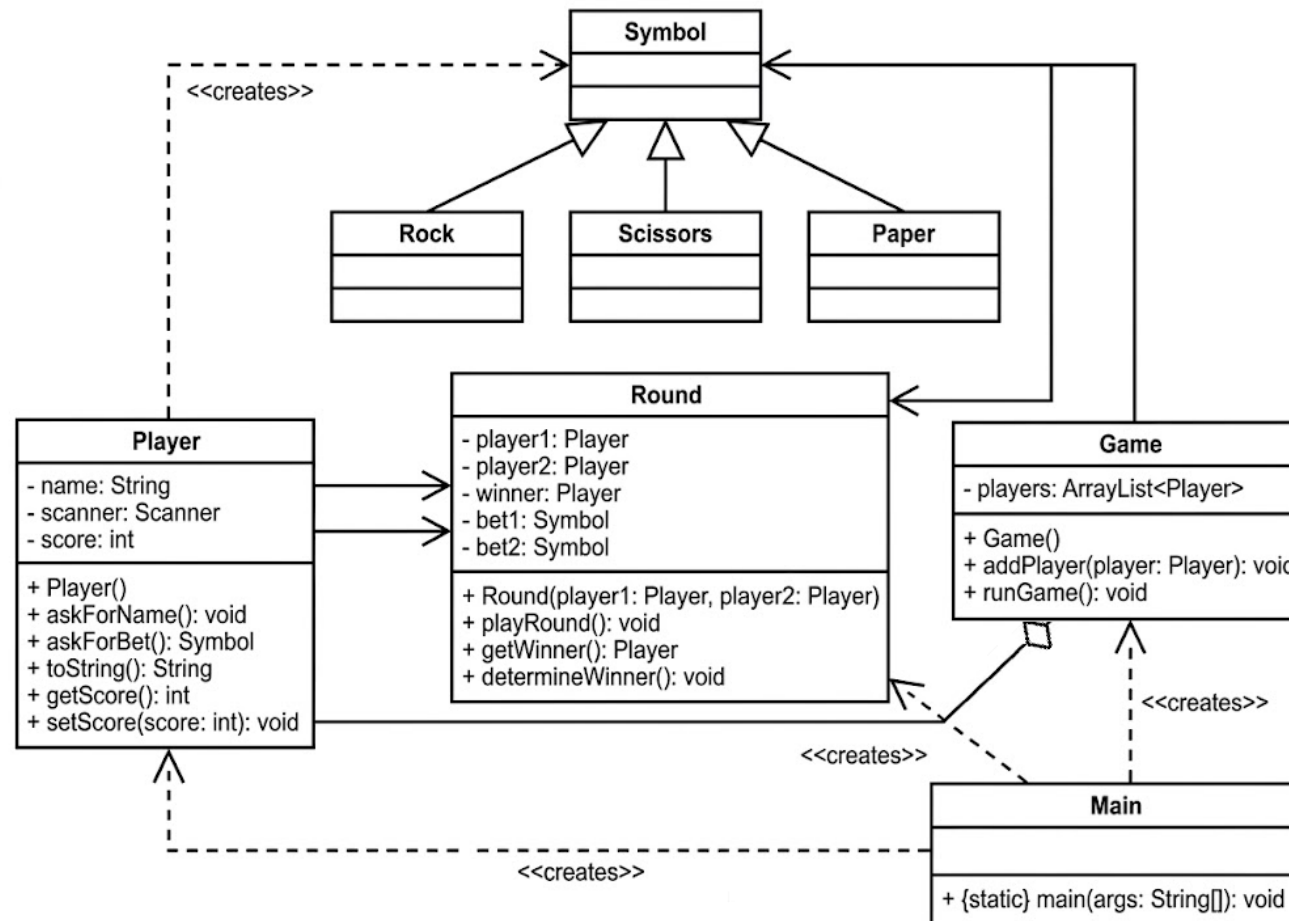
    public void askForName() {
        this.name = this.scanner.nextLine();
    }

    public String toString() {
        return this.name;
    }
}
```

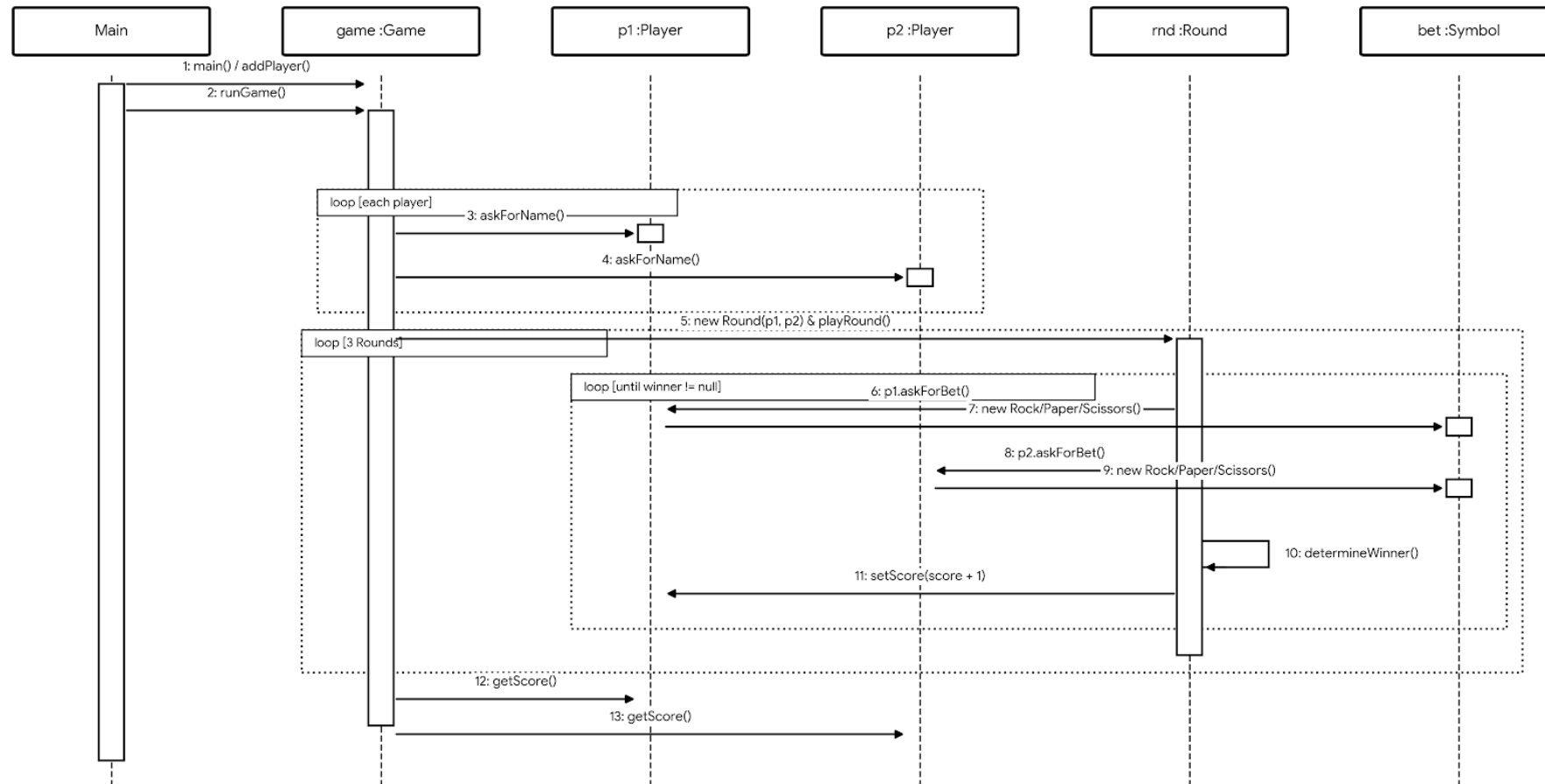
Aufgabe II: Klassen weiterentwickeln

- a) In der Schleife von Game soll jeweils ein neues Runden-Objekt generiert werden. Dazu brauchen wir eine neue Klasse Runde. Diese hat einen Konstruktor **Runde()**, welcher die beiden Spieler als Referenzen entgegennimmt.
- b) Definieren Sie eine Klasse **Symbol**, und drei Unterklassen **Schere**, **Stein** und **Papier**. Diese Klassen haben keine Objektvariablen.
- c) Die Runde soll nun die Spieler fragen, welches Symbol sie "wetten"
- d) Die Runde soll überprüfen wer gewonnen hat. Verwende **getClass()** auf den Symbolen.

Mögliche Lösung: siehe Skeleton auf Teams



Sequenzdiagramm



Ohne OOP

```
import java.util.Scanner;

public class Main {

    public static void main(String[] args) {
        System.out.println("starting up Game Skeleton (2-Player)");
        System.out.println("Game started... waiting for players...");

        Scanner scanner = new Scanner(System.in);

        // Spieler-Namen abfragen
        System.out.println("Player 0 enter your name:");
        String player1 = scanner.nextLine();
        System.out.println("Player 1 enter your name:");
        String player2 = scanner.nextLine();

        int score1 = 0;
        int score2 = 0;

        System.out.println("Starting Game");
        System.out.println("Player 1: " + player1);
        System.out.println("Player 2: " + player2);

        // 3 Runden spielen
        for (int round = 0; round < 3; round++) {
            System.out.println("We are playing round: " + round);
            System.out.println("Starting a new round between " + player1 + " and " + player2);

            String winner = null;
            String bet1 = "";
            String bet2 = "";

            // Schleife läuft, bis es keinen Gleichstand gibt
            while (winner == null) {
                // Zug Spieler 1
                bet1 = "";
                while (!bet1.equals("scissors") && !bet1.equals("rock") && !bet1.equals("paper")) {
                    System.out.println(player1 + ", please enter your bet:");
                    bet1 = scanner.nextLine();
                }
                if (bet1.equals("scissors")) {
                    System.out.println(player1 + " has chosen scissors.");
                } else if (bet1.equals("rock")) {
                    System.out.println(player1 + " has chosen rock.");
                } else if (bet1.equals("paper")) {
                    System.out.println(player1 + " has chosen paper.");
                } else {
                    System.out.println("Invalid bet. Please enter 'scissors', 'rock', or 'paper'.");
                }
            }
        }
    }
}
```

```
// Zug Spieler 2
bet2 = "";
while (!bet2.equals("scissors") && !bet2.equals("rock") && !bet2.equals("paper")) {
    System.out.println(player2 + ", please enter your bet:");
    bet2 = scanner.nextLine();
    if (bet2.equals("scissors")) {
        System.out.println(player2 + " has chosen scissors.");
    } else if (bet2.equals("rock")) {
        System.out.println(player2 + " has chosen rock.");
    } else if (bet2.equals("paper")) {
        System.out.println(player2 + " has chosen paper.");
    } else {
        System.out.println("Invalid bet. Please enter 'scissors', 'rock', or 'paper'.");
    }
}

// Gewinner der Runde ermitteln
if (bet1.equals(bet2)) {
    winner = null;
} else if ((bet1.equals("rock") && bet2.equals("scissors")) ||
           (bet1.equals("paper") && bet2.equals("rock")) ||
           (bet1.equals("scissors") && bet2.equals("paper"))) {
    winner = player1;
} else {
    winner = player2;
}

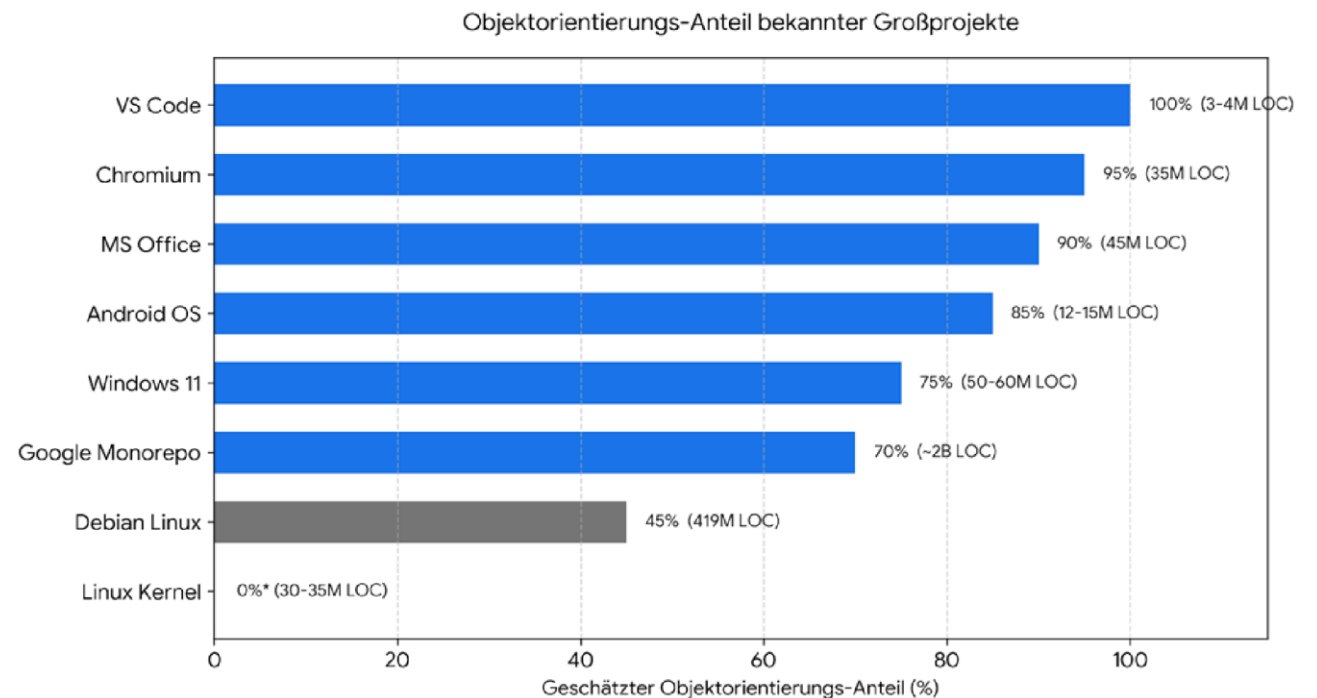
// Punktestand aktualisieren und Ausgabe
if (winner != null && winner.equals(player1)) {
    score1++;
    System.out.println(player1 + " wins the round with " + bet1 + " against " + player2 + "'s " + bet2 + " and now has a score of " + score1);
} else if (winner != null && winner.equals(player2)) {
    score2++;
    System.out.println(player2 + " wins the round with " + bet2 + " against " + player1 + "'s " + bet1 + " and now has a score of " + score2);
} else {
    System.out.println("It's a tie!");
}

System.out.println("Game is over");
System.out.println("Final Scores:");
System.out.println(player1 + ": " + score1);
System.out.println(player2 + ": " + score2);
System.out.println("Game ended.");

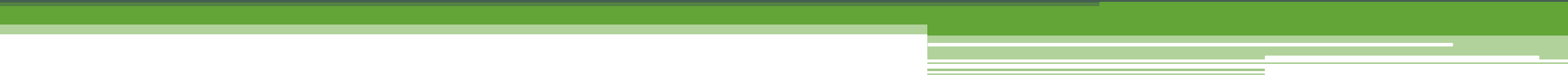
scanner.close();
}
```

Grosse Software Projekte in Lines of Code (LOC)

- Google Monorepo - 2'000'000'000 LOC
- Linux mit allen Bibliotheken (Debian) – 420'000'000 LOC
- Chrome – 35'000'000 LOC
- Windows – 50-60 Mio LOC
- MS Office – 45 Mio LOC



Projekt



Spiel

- Wählt ein einfaches, zugbasiertes Spiel, welches ohne komplizierte Grafik auskommt
- Wählt bestehende Spiele oder erfindet eigene Spiele – nach eurem Gutdünken. Nur ja nicht zu kompliziert.
- Das Spiel sollte sich eignen, Objektorientierung und den Einsatz von Klassen zu erproben

Liste von Spielen

- Memory
- Schiffe Versenken
- Minesweeper
- Monopoly
- Mensch ärgere dich nicht
- Missionare und Kannibalen
- ...

Zielsetzungen des Projekts

Die folgenden Ziele gilt es in dem Projekt zu erreichen:

- Die Schüler*innen schreiben ihre Programme gemeinsam und lernen voneinander.
- Alle Schüler*innen verstehen ihre Programme und können diese erklären und Fragen über ihre Programme beantworten. Sie nutzen das Internet als Rechercheinstrument, aber verstehen genau, was sie in ihre eigenen Programme übernehmen und vermerken solche Stellen.
- Die Schüler*innen können die Grundkonzepte der Objektorientierung in einem eigenen kleinen Computerspiel sinnvoll anwenden

Objektorientierungs-Anforderungen

1. Die Wahl der Klassen und deren Interaktionen machen Sinn
2. Public Modifier für Klassen, Private Modifier für Instanzvariablen
3. Die kennengelernten Konzepte der Objektorientierten Programmierung wurden sinngemäss eingesetzt. Das heisst:
 - a) Mehrere Klassen werden verwendet
 - b) Vererbung wird mindestens einmal eingesetzt
 - c) Methodenüberschreiben wird mindestens einmal eingesetzt

Generelle Anforderungen

- Fehlerhafte Eingaben werden abgefangen. Das Spiel geht entweder einfach weiter oder der Benutzer kann sich korrigieren.
- Es kommt sicher kein Java Stacktrace und ein Programmabbruch aufgrund von Fehlern vor.

Code-Qualität / Schönheit / Ästhetik

- Nutzt die Unterlagen zu Java Code Formattierung, z.B.
 - Wo kommen private Klassenvariablen?
 - Wo kommen Konstruktoren hin?
 - Was ist Lower Camel Case? Was ist Upper Camel Case? etc.
- Verseht euren Code mit Kommentaren wo sinnvoll. Aber ihr müsst es auch nicht übertreiben
- Einrückungen gemäss "common sense"

Code aus dem Internet

- Ihr dürft im Internet recherchieren, wie dieses oder jenes programmiert werden kann – wenn ihr es verstanden habt, könnt ihr diese Quellen dann auch benutzen
- Jede externe Quelle muss angegeben werden – per Kommentar in eurem Code.
- Die Verwendung von ganzen Programmen aus dem Internet ist selbstverständlich nicht gestattet. Es handelt sich dabei um Plagiate. Diese werden geahndet.
- Was ihr abgibt, müsst ihr verstehen. Abgegebener Code, welchen ihr nicht versteht, ist offensichtlich plagiiert und wird geahndet.

Terminlichkeiten

- Das Projekt ist per MS Teams der Lehrkraft zu übermitteln
- Der Abgabetermin ist der **24.11.2026**

Bewertung

- 2 Fragen zu eurem individuellen Projekt in der nächsten Prüfung